float32: save a value using 32 bits (binary digits)

$\underbrace{0\ 1\ 0\ 0\ \cdots\ 0}_{\text{32 in total}}$

(note 8 bits = 1 byte
1024 bytes = 1 kb
1024 kb = 1 mb
1024 mb = 1 Gb
so 1 billion byte ≈ 1 Gb

float16: save using 16 bits

How do we actually save a float number? (e.g. 1234.5678)
(using 32 bits)

□ ↑ first bit
0: positive
1: negative

□□□□□□□□ ↑ 8 bits: range
(decides the largest number float32 can represent)

□□ ⋯ □ ↑ 23 bits: precision
(decides how accurate the representation would be)

(note: tf32 or BF 32 may use different value of range/precision but the total bits add up to 32)

example 1: 1234.5678.

Step 1: it is positive → first bit 0

Step 2: represent integer part: 1234 → 10011010010

Step 3: represent fraction part: .5678 → 1001000101... (go on forever)

$0.5678 \times 2 = 1.1356$

$0.1356 \times 2 = 0.2712$

$0.2712 \times 2 = 0.5424$

So 1234.5678 → 10011010010.1001000101...

→ $1.\underbrace{00110100101001000101\ldots}_{\text{take first 23 into precision}} \times 2^{10}$ → move the points 10 times until only one 1 before

ignored, for any value it would be 1

this 10 + 127 = 137
= 10001001
this goes into range

Q: why do we need to add 127 here?

A: if we store a super small float, e.g. 0.00001002234, the range would be 2^{-17} or something

$-17 + 127 = 110$ this is still positive
→ so we save one bit for storing negative range